Lecture 5. Advanced Topics for Turing Machines

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Review-Definition of Turing Machines

- A Turing machine is a 7-tuple:
  - Q = set of ***states***
  - Σ = the ***input*** alphabet
  - Γ = the ***tape alphabet***, $\Sigma \cup \{\bot\} \subseteq \Gamma$
  - $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L,R\}$ is the ***transition function***
  - $q_{start} \in Q$
  - $q_{accept} \in Q$
  - $q_{reject} \in Q$, $q_{accept} \neq q_{reject}$



Three outcomes

- ***NOTE:*** 3 different outcomes
  - The TM accepts (the input is in the TM's language)
  - The TM rejects
  - The TM runs forever

  } input not in the language

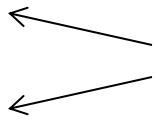
- A T.M. ***M*** recognizes a language ***A*** if
  - L(***M***) = ***A***
- A T.M. ***M*** decides a language ***A*** if
  - L(***M***) = ***A*** *and* ***M*** rejects all w ∉ ***A*** (I.e., it always halts)



The Language A_{TM}

- A_{TM} = {<M,w> | M is a TM and M accepts w}

- A_{TM} is a ***recognizable*** language
  - Simulate M on input w
  - If it accepts, accept
  - If it rejects, rejects
  - (If M loops the simulation will also loop)



Can we distinguish Between these two scenarios?

- A_{TM} is ***not*** a Turing-***decidable*** language



Reduction via computation history

Example 6

$$ALL_{\text{CFG}} = \{ \langle G \rangle : G \text{ is a CFG that generates all strings} \}$$

If ALL_{CFG} can be decided, so can $\overline{A_{\text{TM}}}$.



Reductions via computation histories

- Computation history: sequence of configurations $C_1, C_2, \ldots$
  - So C_{i+1} is the successor of C_i.

- Reduction from A_{TM} to ALL_{CFG}:
  - Construct a CFG G s.t.
  - M accepts w iff G **does not** generate its computation histories
  - Otherwise G generates all strings

- CFG $\Leftrightarrow$ PDA



Reductions via computation histories

- Computation history: $\#C_1\#C_2\#\ldots\#C_k\#$

- We construct a PDA D that accepts strings of computation histories:
  - Do not start with C_1
  - Do not end with an accepting configuration
  - some C_i does not yield C_{i+1} legally

- When M accepts w which means there exists $\#C_1\#C_2\#\ldots\#C_k\#$ satisfying all, the $\overline{L(D)}$ is not empty



- The PDA accepts all incorrect computational histories
  1. if C_1 is not the start configuration
  2. if C_k is not the accepting configuration
  3. if C_i does not yield C_{i+1}

- But remember {w#w} is not context-free
- How to make $\#C_1\#C_2\#\ldots\#C_k\#$ context-free?



- The PDA accepts all incorrect computational histories
  1. if C_1 is not the start configuration
  2. if C_k is not the accepting configuration
  3. if C_i does not yield C_{i+1}

- In fact, the strings that the PDA accepts is arranged as:

  $\# C_1 \# C_2^R \# C_3 \# C_4^R \# C_5 \# C_6^R \# \ldots$

  - it's definitely context-free
  - e.g. $\# C_1 \# C_2^R$: #11 q_1 1100111#111001 q_2 111



PDA D

- Nondeterministically branching to guess which conditions to check, if one of the branches satisfies, accept
  1. if C_1 is not the start configuration
  2. if C_k is not the accepting configuration
  3. if C_i does not yield C_{i+1}
     - the small window is not correct
     - the other part is not correct (e.g. $\{w\#w'^R, w \neq w'\}$)

- First push C_i into the stack
- Pop each symbol and match it with C_{i+1}
- For the first symbol does not match, compare the surrounding symbols to check whether it is a valid transition of the TM
- If there are other mismatches of C_i and C_{i+1}, accept

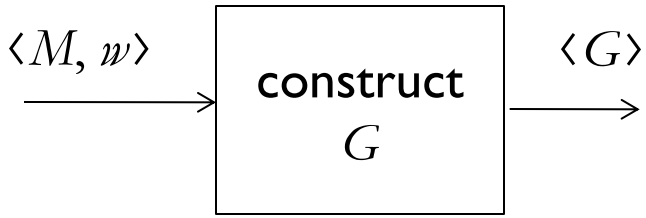


Example 5

$$ALL_{\text{CFG}} = \{ \langle G \rangle : G \text{ is a CFG that generates all strings} \}$$

If ALL_{CFG} can be decided, so can $\overline{A_{\text{TM}}}$.

undecidable



- G accepts all strings except accepting computation histories of (M, w)



For context-free grammars

- Undecidable:
  - Universality: given CFG G, is $L(G)=\Sigma^*$?
  - Equivalence: given CFG G and H, are $L(G)=L(H)$?
  - Ambiguity: is G ambiguous?

- Decidable:
  - Emptiness: is $L(G)=\emptyset$?
  - Finiteness: is L(G) finite?
  - Membership: check if $w \in L(G)$



Time complexity class

- TIME(t(n)): all languages that are decidable by an O(t(n)) time *deterministic* Turing machine
- NTIME(t(n)): all languages that are decidable by an O(t(n)) time *nondeterministic* Turing machine

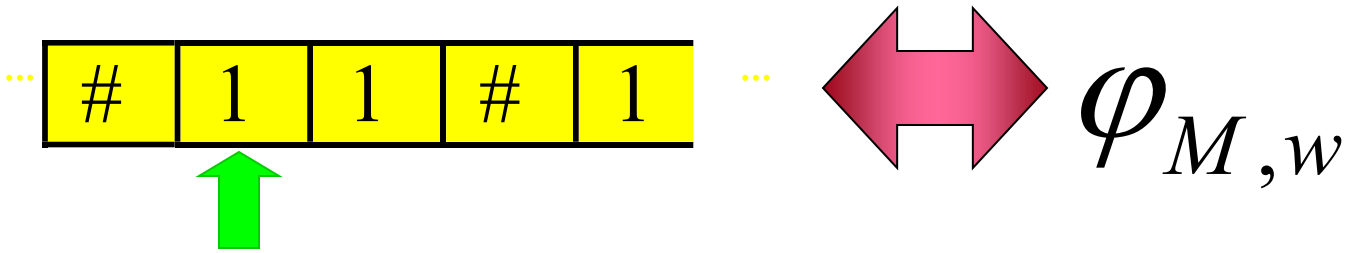
$$P = \bigcup_k TIME(n^k)$$

$$NP = \bigcup_k NTIME(n^k)$$



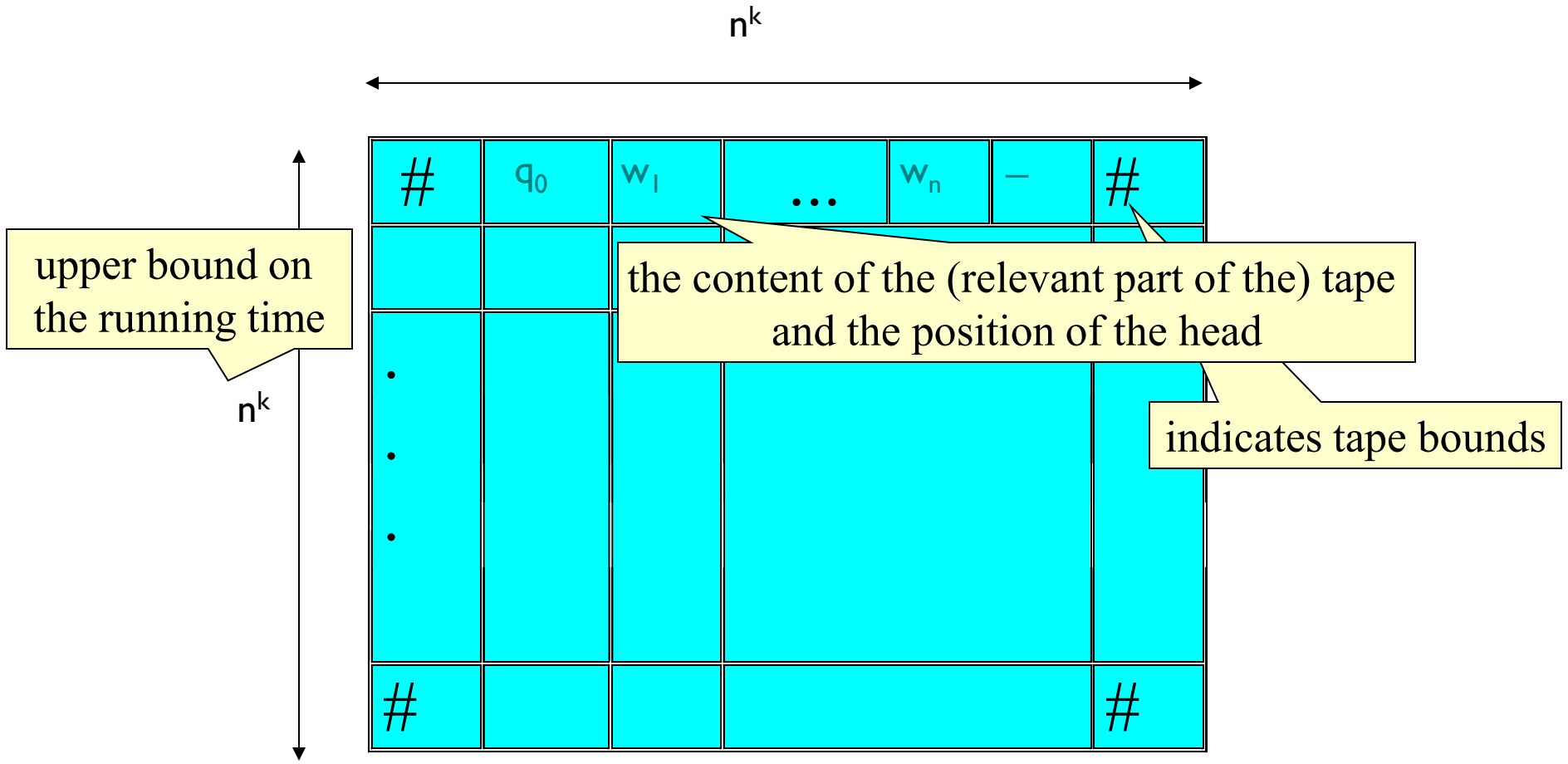
The Cook-Levin Theorem: SAT is NP-Complete

Proof Idea: For any polynomial NTM M and any input string w, we construct a Boolean formula $\varphi_{M,w}$ which is satisfiable iff M accepts w





Representing a Computation by a Configurations Table



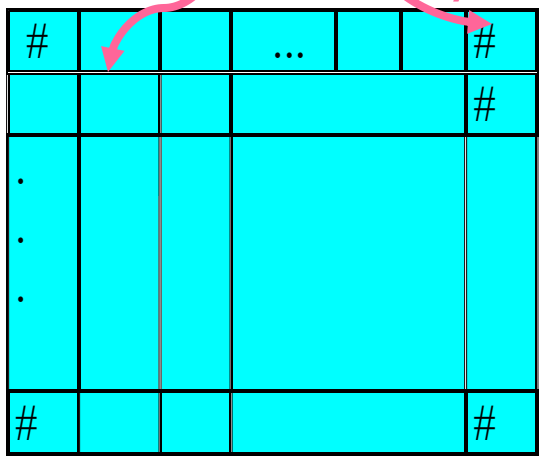
The Variables of the Formula

stands for: "Is s the content of cell (i,j)?"

$x_{i,j,s}$

symbol ($s \in \Gamma \cup Q \cup \{\#\}$)

position in the tableau ($1 \le i,j \le n^k$)





The Formula φ

$$\varphi_{M,w} = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

cell content consistency

input consistency

transition legal

machine accepts

Satisfiable iff there is a branch of the NTM accepts

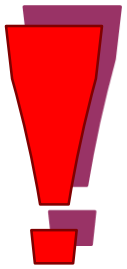


Ensuring Unique Cell Content

$$\varphi_{cell} = \bigwedge_{1 \le i,j \le n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s \neq t \in C} \left(\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}} \right) \right) \right]$$

The (i,j) cell must contain some symbol

It shouldn't contain different symbols.



Note: the length of this formula is polynomial in n



Ensuring Initial Configuration Corresponds to Input

Observe: we can explicitly state the desired configuration in the first step. Assuming the input string is $w_1 w_2 \ldots w_n$,

$$\varphi_{start} = X_{1,1,\#} \wedge X_{1,2,q_0} \wedge X_{1,3,w_1} \wedge \ldots \wedge X_{1,n+3,_} \wedge \ldots \wedge X_{1,n^k-1,_} \wedge X_{1,n^k,\#}$$



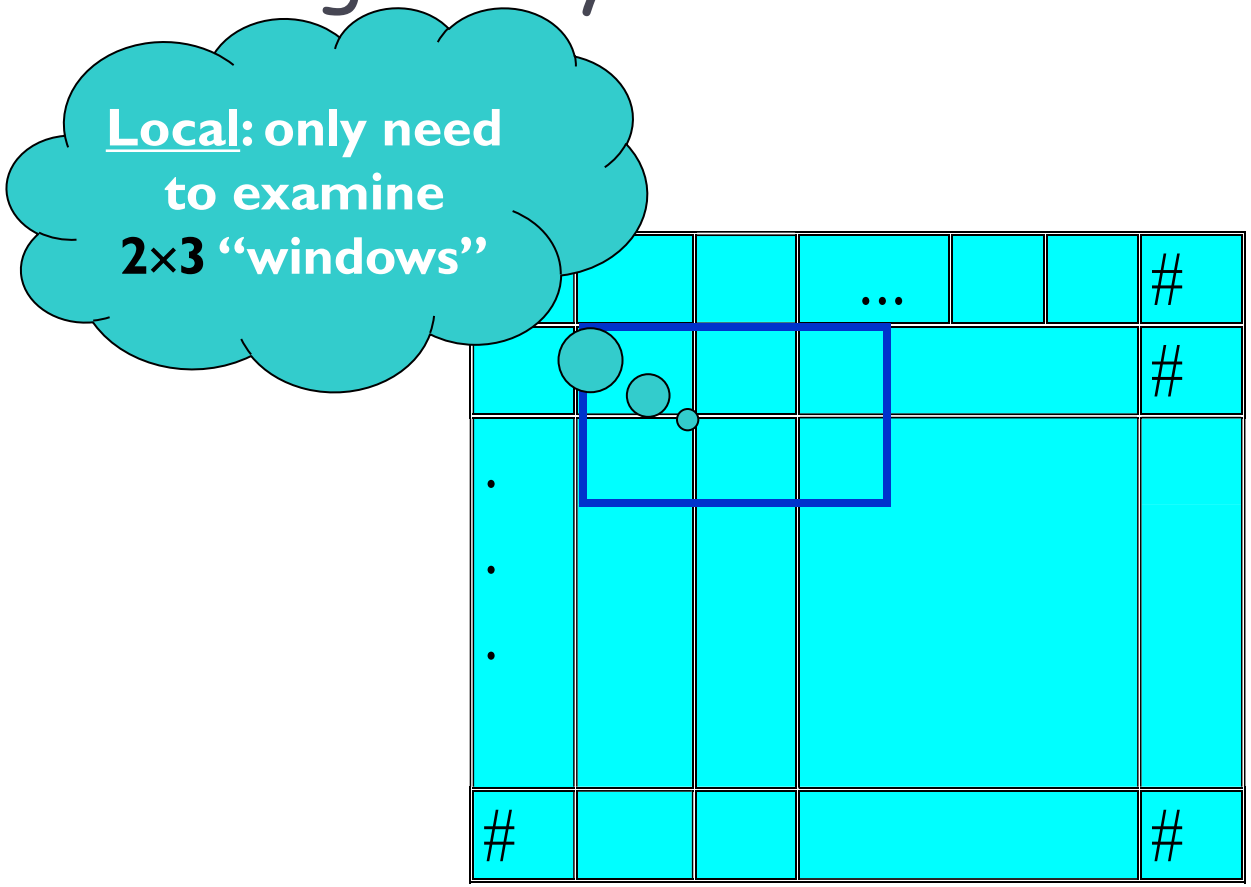
Ensuring the Computation Accepts

The accepting state is visited during the computation.

$$\varphi_{accept} = \bigvee_{1 \le i,j \le n^k} x_{i,j,q_{accept}}$$



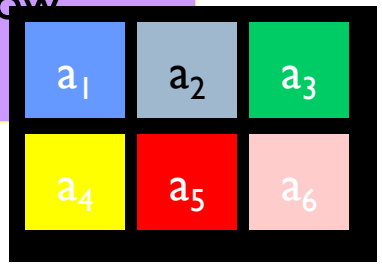
Ensuring Every Transition is Legal



Ensuring Every Transition is Legal

$$\varphi_{move} = \bigwedge_{1 \le i,j \le n^k} \bigvee_{a_1,\ldots,a_6} \left(x_{i-1,j,a_1} \wedge \ldots \wedge x_{i+1,j+1,a_6} \right)$$

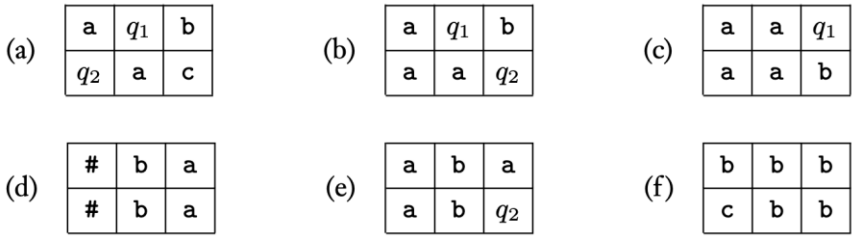
for any $a_1,\ldots,a_6$ s.t. this is a legal window





Ensuring Every Transition is Legal

$$\varphi_{move} = \bigwedge_{1 \le i,j \le n^k} \bigvee_{a_1,\ldots,a_6} \left(x_{i-1,j,a_1} \wedge \ldots \wedge x_{i+1,j+1,a_6} \right)$$



Example of legal windows

NOTE: It is nondeterministic,
So many possibilities for legal windows



The Bottom Line

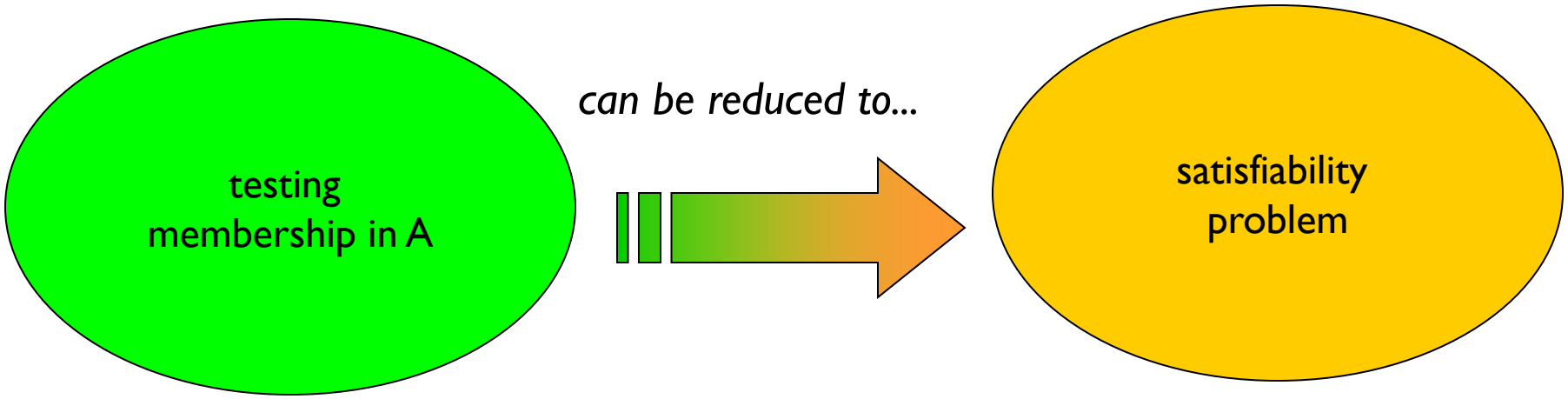
$$\varphi_{M,w} = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

φ, which is of size polynomial in n is satisfiable iff the TM accepts the input string.



Conclusion: SAT is NP-Complete

For any language *A* in NP,





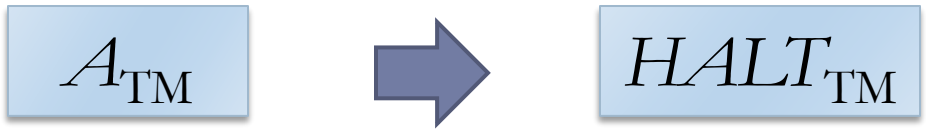
Turing and Mapping Reducibility

Turing Reducibility

- Suppose you have an "oracle" which decides a language B
- Oracle Turing machine: a TM capable of querying an oracle
- M^B: the oracle TM that has an oracle for language B
- Language A is Turing reducible to B, if an oracle TM M^B decides A, write $A \leq_T B$



- We just assume there is an oracle for HALT_{TM}
- Then we can solve A_{TM}, so A_{TM} is "no harder" than HALT_{TM}



- From A_{TM} is undecidable, HALT_{TM} is also undecidable



Recall reduction for NP-completeness

- Language L polynomial-time reduces to L' if
  - there exists a polynomial-time computable map R that takes an instance x of L into instance $R(x)$ of L' s.t.

$x \in L$ if and only if $R(x) \in L'$

We write $L \leq_p L'$



Example: IS $\leq_p$VC

$IS = \{(G, k)$: G is a graph with an independent set of k vertices$\}$

$VC = \{(G, k)$: G is a graph with a vertex cover of size $k\}$

IS is NP-hard  VC is NP-hard



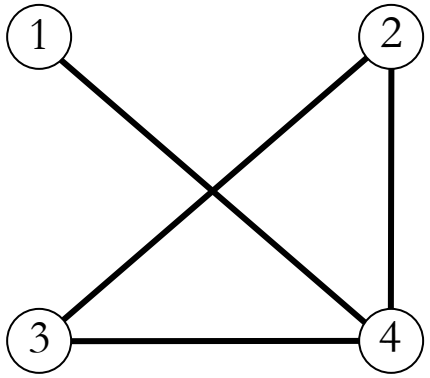
Example: IS $\leq_p$VC

$IS = \{(G, k)$: G is a graph with an independent set of k vertices$\}$

$VC = \{(G, k)$: G is a graph with a vertex cover of size $k\}$

An independent set is a subset of vertices so that no pair is connected

$\{1, 2\}, \{1, 3\}, \{4\}$ are independent sets



A vertex cover is a set of vertices that touches (covers) all edges

$\{2, 4\}, \{3, 4\}, \{1, 2, 3\}$ are vertex covers



Reducing IS to VC

- Claim

  S is an independent set of G if and only if
  $\overline{S}$ is a vertex cover of G

- Proof

S is an independent set of G

⇕

no edge has both endpoints in S

⇕

every edge has an endpoint in $\overline{S}$

⇕

$\overline{S}$ is a vertex cover of G

So instance (G,k) of IS is mapped to instance (G,n-k) of VC



What about…

- Polynomial-time Turing reduction
- Assume we have an oracle for Vertex Cover, then we can construct a polynomial-time algorithm for Independent Set

```
procedure IS(G, k) {
        …
        call VC(G',k');
        …
}
```

- So if we can solve VC in polynomial time, we can also solve IS

What's the difference?



- Both SAT and NON-SAT can be (polynomial-time) Turing reducible to each other

- But not for polynomial-time reducibility!
  - we cannot map a co-NP instance to a NP instance

> Language L polynomial-time reduces to L' if there exists a polynomial-time computable map R that takes an instance x of L into instance $R(x)$ of L' s.t. $x \in L$ if and only if $R(x) \in L'$



Mapping reducibility

- A function f is a *recursive function* if some Turing machine M, on every input w, halts with just f(w) on its tape
  - or with a write-only printer

- Language A is *mapping reducible* to language B, if there is a recursive function f, such that $w \in A \Leftrightarrow f(w) \in B$
  - denoted $A \leq_m B$
  - The function f is called the reduction of A to B



Mapping reducibility

- If $A \leq_m B$ and B is decidable, then A is decidable
  - same as Turing reducibility

- If $A \leq_m B$ and B is Turing-recognizable, then A is Turing-recognizable
  - suppose M is a recognizer of B, and f is the reduction
  - the recognizer of A is M(f(w))

- We can use the mapping reducibility to prove Turing-recognizability



Example

- EQ_{TM}={<M_1,M_2>| M_1 and M_2 are TMs and $L(M_1)=L(M_2)$}

- By Turing reducibility, we show that EQ_{TM} is undecidable
  - *Recall:* $E_{\text{TM}} = \{ \langle M \rangle : M$ is a TM that accepts no input$\}$ is undecidable
  - When given M, we can construct a TM M' which rejects all inputs, and check whether <M,M'>$\in EQ_{TM}$



Example

- $EQ_{TM}=\{<M_1,M_2>|\ M_1$ and M_2 are TMs and $L(M_1)=L(M_2)\}$
- By Turing reducibility, we show that EQ_{TM} is undecidable
- By mapping reducibility, we show that EQ_{TM} is neither recognizable nor co-recognizable
  - reduce A_{TM} to both EQ_{TM} and $\overline{EQ_{TM}}$

$A_{TM} = \{<M,w> \mid M$ is a TM and M accepts $w\}$
A_{TM} is not co-recognizable



- From A_{TM} to $\overline{EQ_{TM}}$, reducing function f:

```
f(<M,w>)                                  //input for A_TM
        Define M_1 which rejects all inputs
        Define M_2 which simulates M on input w
        Output <M_1,M_2>                  //input for ¬EQ_TM
```



- From A_{TM} to $\overline{EQ_{TM}}$, reducing function f:

```
f(<M,w>)                                  //input for A_TM
        Define M_1 which rejects all inputs
        Define M_2 which simulates M on input w
        Output <M_1,M_2>                  //input for ¬EQ_TM
```

- So $L(M_1) \neq L(M_2)$ iff M accepts w
  - that's a mapping reduction from A_{TM} to $\overline{EQ_{TM}}$
  - thus also a reduction from $\overline{A_{TM}}$ to EQ_{TM}
- Since $\overline{A_{TM}}$ is not recognizable, EQ_{TM} is not recognizable



- From $\overline{A_{TM}}$ to $\overline{EQ_{TM}}$, reducing function f':

```
f'(<M,w>)                                  //input for A_TM
        Define M_1 which accepts all inputs
        Define M_2 which simulates M on input w
        Output <M_1,M_2>                   //input for EQ_TM
```

- So $L(M_1)=L(M_2)$ iff M accepts w
- Since $\overline{A_{TM}}$ is not recognizable, $\overline{EQ_{TM}}$ is not recognizable



Difference of Turing-reduction and mapping-reduction

- Mapping reducibility is sensitive to complementation while Turing reducibility is not



Difference of Turing-reduction and mapping-reduction

- Mapping reducibility is sensitive to complementation while Turing reducibility is not

$$E_{\mathrm{TM}} = \{\ \langle M \rangle\ : M \text{ is a TM that accepts no input}\}$$

- Recall the Turing reduction $\mathrm{SOME}_{\mathrm{TM}} \leq_T E_{\mathrm{TM}}$:

- In fact $\mathrm{SOME}_{\mathrm{TM}} \leq_m \overline{E_{\mathrm{TM}}}$ since they are equal,
- But $\mathrm{SOME}_{\mathrm{TM}}$ cannot be mapping reducible to E_{TM} since
  - $\overline{E_{\mathrm{TM}}}$ is Turing-recognizable but $\overline{\mathrm{SOME}_{\mathrm{TM}}}$ is not Turing-recognizable



- Thus, Turing-reduction only for proving undecidability, and mapping-reduction can be used for proving unrecognizability
- Mapping-reduction is also called "many-one reduction"
- If the recursive function f(w) is a one-to-one function, then it is a "one-one reduction"



For NP-completeness

- For proving NP-completeness, we use polynomial-time many-one reductions *(Karp reduction)*:
  - polynomial-time algorithm for transforming inputs to problem *A* into inputs to problem *B*, such that the transformed problem has the same output as the original problem

- There are also polynomial-time Turing reductions *(Cook reduction)*
  - *an algorithm that solves problem A using a polynomial number of calls to a subroutine for problem B*

It's still open whether they can define the same class of NP-completeness!



A paper conjecturing they are different:

Electronic Colloquium on Computational Complexity, Report No. 126 (2014)

ECCC

Separating Cook Completeness from Karp-Levin Completeness under a Worst-Case Hardness Hypothesis

Debasis Mandal[*] A. Pavan[*] Rajeswari Venugopalan[†]

Abstract

We show that there is a language that is Turing complete for NP but not many-one complete for NP, under a *worst-case* hardness hypothesis. Our hypothesis asserts the existence of a non-deterministic, double-exponential time machine that runs in time $O(2^{2^{n^c}})$ (for some $c > 1$) accepting Σ^* whose accepting computations cannot be computed by bounded-error, probabilistic machines running in time $O(2^{2^{\beta 2^{n^c}}})$ (for some $\beta > 0$). This is the first result that separates completeness notions for NP under a worst-case hardness hypothesis.

Recursion Theorem

The recursion theorem

- Living things are machines *(Materialism)*
- Living things can self-reproduce
- ∴ Some machines can self-reproduce





The recursion theorem

- Living things are machines
- Living things can self-reproduce
- ∴ Some machines can self-reproduce
  - e.g. Computer virus

Maybe beyond intuition, since usually we need more complex machines to construct machines





Recursion theorem

- Recall the recursive function on natural numbers **N**→**N**

- Each Turing machine M has a natural number code <M>
  - M has infinite many equivalent TMs with different codes

M is equivalent to M' when:
- M accepts w ⇔ M' accepts w
- M rejects w ⇔ M' rejects w
- M loops on w ⇔ M' loops on w



Recursion theorem

- Let t: **N**→**N** be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F



Recursion theorem

- Let t: **N**→**N** be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F
- Proof.
  - define a Turing machine D, whose input and output are codes of TMs

```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                d=M(<M>);
                run the machine with code d on y;
        }
}
```



Recursion theorem

- Let t: **N**→**N** be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F
- Proof.
  - define a Turing machine D, whose input and output are codes of TMs

```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

  - Note that D is recursive (always halts), since running D doesn't need to run any part of G(y)



Recursion theorem

- Let t: **N**→**N** be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F

```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

  - also define a TM V:

```
V(<M>) {
        output t(D(<M>));
}
```



Recursion theorem

- Let t: **N**→**N** be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F

```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

  - also define a TM V:

```
V(<M>) {
        output t(D(<M>));
}
```

What about the machine D(<V>)?



- Let t: **N**→**N** be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F

```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

```
V(<M>) {
        output t(D(<M>));
}
```

```
Let <F>=D(<V>), then <F> is the code of function:
F(y)
{
        run the machine with code V(<V>) on y;
}
```

- (Note that t and D are both recursive, so V is also recursive)



- Let t: **N**→**N** be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F

```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

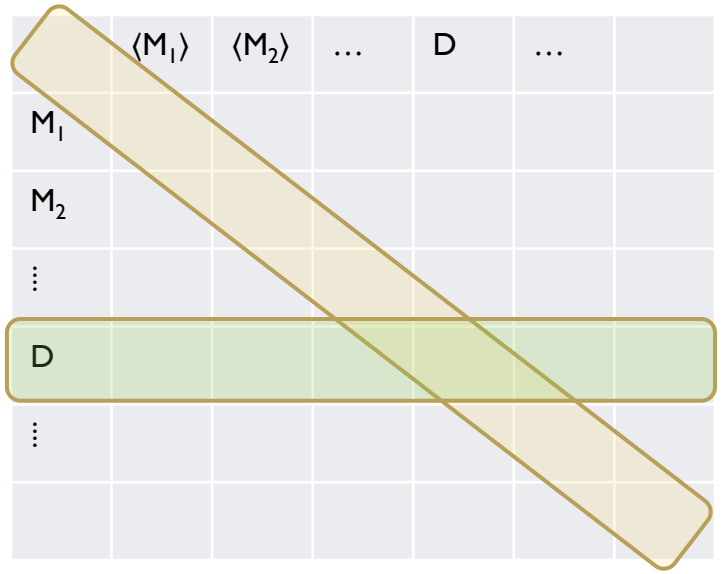
```
V(<M>) {
        output t(D(<M>));
}
```

```
Let <F>=D(<V>), then <F> is the code of function:
F(y)
{
        run the machine with code V(<V>) on y;
}
But V(<V>)=t(D(<V>))=t(<F>)
So running F(y) is equivalent as simulating the TM
with code t(<F>) on y
```



Diagonalized proof

- But not for contradiction!
- The TM D corresponds to the diagonal and a line

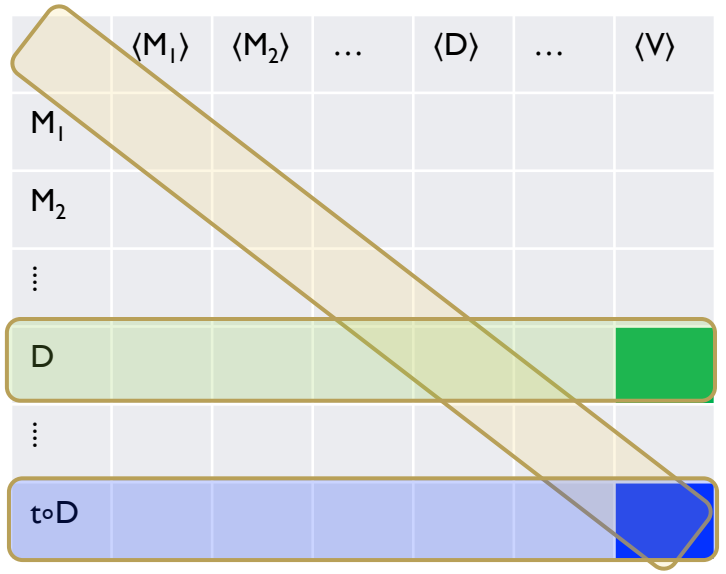



```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```



Diagonalized proof

- But not for contradiction!
- The TM D corresponds to the diagonal and a line
- V=t∘D corresponds to another line
- so D(<V>) is equivalent to t(D(<V>))




```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

```
V(<M>) {
        output t(D(<M>));
}
```



Kleene's *s-m-n* Theorem

- Suppose we have a TM $M(x_1,\ldots,x_m, y_1,\ldots,y_n)$ whose input is composed of m+n variables

- There is a TM $T(\langle M\rangle, x_1,\ldots,x_m)$ when given $M, x_1,\ldots,x_m$, outputs a TM M' such that:
  - $M'(y_1,\ldots,y_n)= M(x_1,\ldots,x_m, y_1,\ldots,y_n)$
  - (Simply put $x_1,\ldots,x_m$ into the machine M)

Note that the output of T is a Turing machine



Kleene's recursion theorem

- Let t be a recursive function t: $\mathbf{N} \times \mathbf{N} \rightarrow \mathbf{N}$. There is a Turing machine R: $\mathbf{N} \rightarrow \mathbf{N}$, where

$$R(x)=t(<R>, x)$$

  - (a generalization of the previous theorem)



Kleene's recursion theorem

- Let t be a recursive function t: $\mathbf{N} \times \mathbf{N} \rightarrow \mathbf{N}$. There is a Turing machine R: $\mathbf{N} \rightarrow \mathbf{N}$, where

$$R(x)=t(<R>, x)$$

  - (a generalization of the previous theorem)

Recursion theorem Let t: $\mathbf{N} \rightarrow \mathbf{N}$ be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F



Kleene's recursion theorem

- Let t be a recursive function t: $\mathbf{N} \times \mathbf{N} \rightarrow \mathbf{N}$. There is a Turing machine R: $\mathbf{N} \rightarrow \mathbf{N}$, where

$$R(x)=t(\langle R \rangle, x)$$

Recursion theorem Let t: $\mathbf{N} \rightarrow \mathbf{N}$ be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F

- Define the recursive function s: $\mathbf{N} \rightarrow \mathbf{N}$: (by s-m-n theorem)

```
s(x) {
        output the code of the TM on input y computing t(x,y);
}
```

We treat t(x,·) as a machine with one input



Kleene's recursion theorem

- Let t be a recursive function t: $\mathbf{N} \times \mathbf{N} \rightarrow \mathbf{N}$. There is a Turing machine R: $\mathbf{N} \rightarrow \mathbf{N}$, where

  R(x)=t(<R>, x)

> **Recursion theorem** Let t: $\mathbf{N} \rightarrow \mathbf{N}$ be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F.

- Define the recursive function s: $\mathbf{N} \rightarrow \mathbf{N}$: (by s-m-n theorem)

```
s(x) {
        output the code of the TM on input y computing t(x,y);
}
```

- Then by the recursion theorem on s, we get the TM R
- R(y) = "simulating the TM with code s(<R>) on y" = t(<R>, y)



Kleene's recursion theorem

- Let t be a recursive function t: $\mathbf{N} \times \mathbf{N} \rightarrow \mathbf{N}$. There is a Turing machine R: $\mathbf{N} \rightarrow \mathbf{N}$, where

  R(x)=t(<R>, x)

- This theorem states that a TM can obtain its own description and computes with it
  - So when writing programs, we can obtain its own codes

```
SELF(x):
        Obtain, via the recursion theorem, own description <SELF>
        …
```

  - Then there is a function R(x) which is equivalent to SELF(<R>,x)



Application I

- Another proof of A_{TM} is undecidable:
  - Suppose H(<M,w>) decides A_{TM}:

B(w):

Obtain, via the recursion theorem, own description <B>
Run H(<B,w>)
Accept if H rejects, reject if H accepts



Application II

- Define MIN_{TM} ={<M>| M is a "minimal" TM, that is, no TM with a shorter encoding recognizes the same language}
- Prove MIN_{TM} is not recursively enumerable



Application II

- Define MIN_{TM} ={<M>| M is a "minimal" TM, that is, no TM with a shorter encoding recognizes the same language}
- Prove MIN_{TM} is not recursively enumerable

- Proof.
  - Assume MIN_{TM} is r.e., then there is an enumerator E which lists all strings in it

```
R(w):
        Obtain <R>;
        Run E, producing list <M1>, <M2>, ... of all minimal TMs
        until you find some <Mi> with |<Mi>| strictly greater than |<R>|;
        Return Mi(w);
```

  - Then R is equivalent to M_i, but shorter than M_i, a contradiction



A “quine”

- A **quine** is a non-empty computer program which takes no input and produces a copy of its own source code as its only output
  - a self-reproducing program

- By recursion theorem, we can write a quine:

```
Q {
        print the code of Q;
}
```



Recursion theorem

- Let t be a recursive function. There is a TM F for which t(<F>) is the code of a Turing machine equivalent to F

- let t(x)="PRINT x"
- Turing machine F is equivalent to t(<F>), which outputs <F>
- so F outputs <F>

- You can read another proof of the recursion theorem in the textbook



```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

```
V(<M>) {
        output t(D(<M>));
}
```

```
t(x)="PRINT x", let <F>=D(<V>)
```

```
V(x): print "V(x): "+x+"run V(""+x+"")"
run V("print "V(x)"+x+"run V("+x+")")")
```

Still not easy to satisfy real-world grammar



```
D(<M>) {
        output the code of the Turing machine G:
        G(y) {
                run the machine with code M(<M>) on y;
        }
}
```

```
V(<M>) {
        output t(D(<M>));
}
```

```
t(x)="PRINT x", let <F>=D(<V>)
```

```
V(x): print "V(x): "+x+"run V(""+x+"")"
run V("print "V(x)"+x+"run V("+x+")")")
```

a JAVA quine:

```java
public class Quine
{
  public static void main(String[] args)
  {
    char q = 34;      // Quotation mark character
    String[] l = {    // Array of source code
    "public class Quine",
    "{",
    "  public static void main(String[] args)",
    "  {",
    "    char q = 34;      // Quotation mark character",
    "    String[] l = {    // Array of source code",
    "    ",
    "    };",
    "    for (int i = 0; i < 6; i++)           // Print opening code",
    "        System.out.println(l[i]);",
    "    for (int i = 0; i < l.length; i++)    // Print string array",
    "        System.out.println(l[6] + q + l[i] + q + ',');",
    "    for (int i = 7; i < l.length; i++)    // Print this code",
    "        System.out.println(l[i]);",
    "  }",
    "}",
    };
    for (int i = 0; i < 6; i++)           // Print opening code
        System.out.println(l[i]);
    for (int i = 0; i < l.length; i++)    // Print string array
        System.out.println(l[6] + q + l[i] + q + ',');
    for (int i = 7; i < l.length; i++)    // Print this code
        System.out.println(l[i]);
  }
}
```



“Cheating” quine

- Quines, per definition, cannot receive *any* form of input, including reading a file, which means a quine is considered to be “cheating” if it looks at its own source code
  - in modern computers, the program is also put into the memory, so it’s possible for the program to read the code of itself

```
cat $0
```

```
function a() {
    document.write(a, "a()");
}
a()
```

```
MOV AH,09
MOV DX,0100
INT 21
MOV AH,02
MOV DL,23
INC DL
INT 21
MOV AH,4C
INT 21
DB ‘$’
```



Kolmogorov Complexity

Kolmogorov complexity

- A definition of information

- Data compression:
  - A=0101010101010101010101010101010101
  - B=1110010110100011101010000111010011

  - Of course, A contains less information
  - So A can be compressed





Kolmogorov complexity

- A definition of information
  - Shortest possible description of data
  - Shortest Turing machine generates it?

- Data compression:
  - A=0101010101010101010101010101010101
  - B=1110010110100011101010000111010011

  - Of course, A contains less information
  - So A can be compressed



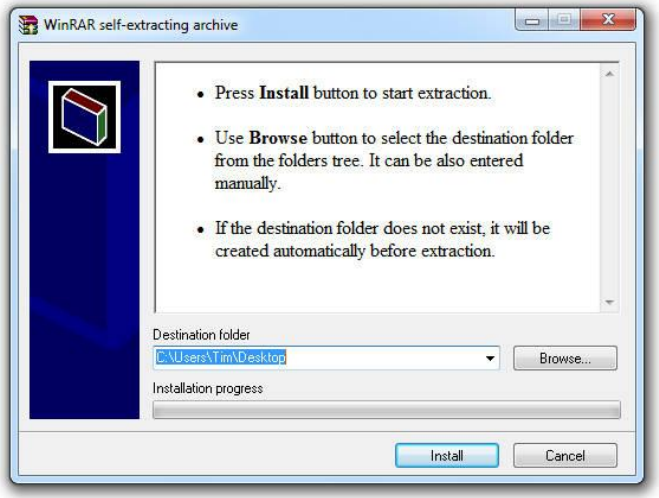
Definition

- Let x be a binary string
- The *minimum description* d(x) of x is the shortest string <M,w> where TM M on input w halts with x on its tape
- The *Kolmogorov complexity* K(x)=|d(x)|



Definition

- Let x be a binary string
- The *minimum description* d(x) of x is the shortest string <M,w> where TM M on input w halts with x on its tape
- The *Kolmogorov complexity* K(x)=|d(x)|
  - Can be seen as a "self-extracting" program





Some property

- For all x, $K(x) \leq |x| + c$, for some constant c
  - We just need a null TM (halts immediately), so $M(x)=x$



Some property

- For all x, $K(x) \leq |x| + c$, for some constant c
  - We just need a null TM (halts immediately), so $M(x)=x$

- For all x, $K(xx) \leq |x| + c$, for some constant c
  - We need a TM copying the input, so $M(x)=xx$



Some property

- For all x, $K(x) \leq |x| + c$, for some constant c
  - We just need a null TM (halts immediately), so $M(x)=x$
- For all x, $K(xx) \leq |x| + c$, for some constant c
  - We need a TM copying the input, so $M(x)=xx$
- For all x, $K(x^n) \leq |x| + c \cdot \log n$, for some constant c



Some property

- For all x, $K(x) \leq |x| + c$, for some constant c
  - We just need a null TM (halts immediately), so $M(x)=x$
- For all x, $K(xx) \leq |x| + c$, for some constant c
  - We need a TM copying the input, so $M(x)=xx$
- For all x, $K(x^n) \leq |x| + c \cdot \log n$, for some constant c
  - a TM M such that $M(x,n)=x^n$
  - the length of the integer n is bounded by $O(\log n)$
  - is it corret: $K(x^n) \leq |x| + \log n + c$?

    alphabet size does not increase, usually just {0,1}



c-compressible

- A string x is *c-compressible* if $K(x) \leq |x| - c$
- If x is not *1-compressible*, we say that x is *incompressible*



Incompressible strings

- Incompressible strings of every length exist



Incompressible strings

- Incompressible strings of every length exist

Proof: (Number of binary strings of length n) = 2^n
and (Number of descriptions of length < n)
$\leq$ (Number of binary strings of length < n)
$= 1 + 2 + 4 + \cdots + 2^{n-1} = 2^n - 1$

Therefore there's at least one n-bit string that does not have a description of length < n



Incompressible strings

- For all n and c,

$$\Pr_{x\in\{0,1\}^n}[K(x)\geq |x|-c]\geq 1-\frac{1}{2^c}$$

- "Most strings are very incompressible"
  - Can be seen as "random strings"



Incompressible strings

- For all n and c,

$$\Pr_{x\in\{0,1\}^n}[K(x)\geq |x|-c]\geq 1-\frac{1}{2^c}$$

- "Most strings are very incompressible"
  - Can be seen as "random strings"

Proof: (Number of binary strings of length n) = 2^n
and (Number of descriptions of length < n-c)
$\leq$ (Number of binary strings of length < n-c)
= $2^{n-c} - 1$
So the probability that a *random* x satisfies
K(x) < n-c
is at most $(2^{n-c} - 1)/2^n < 1/2^c$.



- What about writing a short TM generating strings:
  - **0100011011000001010011100101110111 0000**
  - **1235813213455891442333776109 87**
  - **1262412072050404032036288036288 00**
- Maybe random, maybe not
- Hard in general



Compressibility

- COMPRESS={(x,c)| K(x)≤c}
- Theorem: COMPRESS is undecidable





Compressibility

- COMPRESS={(x,c)| K(x)≤c}
- Theorem: COMPRESS is undecidable

- Proof: Suppose COMPRESS is decidable,

  Construct a machine M:

```
M(n)
        For all y in lexicographical order
                If (y,n)∉COMPRESS, print y and halt
```



Compressibility

- COMPRESS={(x,c)| K(x)≤c}
- Theorem: COMPRESS is undecidable

- Proof: Suppose COMPRESS is decidable,
  Construct a machine M:

```
M(n)
        For all y in lexicographical order
                If (y,n)∉COMPRESS, print y and halt
```

Since (y,n)∉COMPRESS, K(y)>n

(y is the first string of Kolmogorov complexity >n)

But M(n) prints y, so K(y)≤ |<M,n>|=c+log n, a contradiction for large enough n



Chaitin's incompleteness theorem

- Suppose in a formal system F:
  - Any mathematical statement describable in English can also be described within F
  - Proofs are convincing: given (S,P), it is decidable to check that P is a proof of S in F



Chaitin's incompleteness theorem

- Suppose in a formal system F:
  - Any mathematical statement describable in English can also be described within F.
  - Proofs are convincing: given (S,P), it is decidable to check that P is a proof of S in F

- Then for large enough L, for all x, K(x)>L is not provable



Proof

- Suppose we can prove K(x)>L for some x
- Pick the smallest such proof P which proves K(y)>L



Proof

- Suppose we can prove K(x)>L for some x
- Pick the smallest such proof P which proves K(y)>L

Proofs	x↓
1100111001010011001	010000110
111001010010l0	100100001
0100101010010l0l0l101	01001010010
100100100100110010	111000010010



Proof

- Suppose we can prove K(x)>L for some x
- Pick the smallest such proof P which proves K(y)>L

Proofs ↓	x
1110010100101 0 (P)	100100001 (y)
1001001001001 10010	111000010010
1100111001010011001	010000110
0100101010010101011101	01001010010



Proof

- Suppose we can prove K(x)>L for some x
- Pick the smallest such proof P which proves K(y)>L

- Consider the machine:

```
R(L)
        For all list of w.f. Q in lexicographical order
                Check whether Q is a valid argument
                Check whether the last statement of Q is K(y)>L
                If yes, output y and halts
```



Proof

- Suppose we can prove K(x)>L for some x
- Pick the smallest such proof P which proves K(y)>L

- Consider the machine:

```
R(L)
        For all list of w.f. Q in lexicographical order
                Check whether Q is a valid argument
                Check whether the last statement of Q is K(y)>L
                If yes, output y and halts
```

  - it's a machine generating y where |<R,L>|=c+log L



Chaitin's incompleteness theorem

- Then for large enough L, for all x, K(x)>L is not provable

- Berry paradox:

  "The smallest positive integer not definable in under eleven words"

- If we randomly pick a string x of length L+100, then with probability $1-1/2^{99}$, K(x)>L

We can *randomly generate statements in F* which are true but have no proofs in *F with high probability!*



Turing Degree

Turing degree

- Now we use the halting problem to prove undecidability
  - Reduction from A_{TM} ($HALT_{TM}$) to that problem
  - Usually we can also reduce them to A_{TM}
  - So they are equivalent

- For example, the Post correspondence problem
  - We reduced A_{TM} to PCP
  - Trivial to reduce PCP to A_{TM} (just check all possibilities)

a ab	baa a	bab ε	c ab	c ab	bab cc	a baba



Turing degree

- Now we use the halting problem to prove undecidability
  - Reduction from A_{TM} ($HALT_{TM}$) to that problem
  - Usually we can also reduce them to A_{TM}
  - So they are equivalent

- For example, the Post correspondence problem
  - We reduced A_{TM} to PCP
  - Trivial to reduce PCP to A_{TM} (just check all possibilities)
  - So PCP and A_{TM} are Turing equivalent

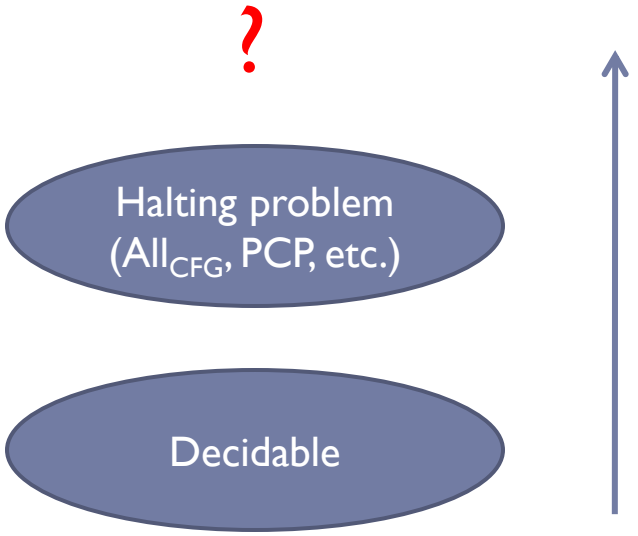


Turing degree

- A Turing degree is such an equivalent class

- We have known two Turing degrees:
  - 0: decidable problems
  - 0': problems equivalent to the Halting problem

  - is there any degree above them?





Superhalting problem

- SUPERHALT is the halting problem for TMs with $HALT_{TM}$ oracles
- SUPERHALT = $\{\langle M^{HALT}, w\rangle \mid \text{TM } M^{HALT} \text{ halts on input } w\rangle\}$
- The Turing degree of it is denoted by 0”



Arithmetical Hierarchy

- For $i \geq 1$, a language L is in $\mathbf{\Sigma_i}$ if there exists a TM M that always halts s.t.

$$x \in L \Leftrightarrow (\exists y_1)(\forall y_2)(\exists y_3)\ldots(Q_i y_i)[M(x,y_1,y_2,\ldots,y_i) \text{ accepts}]$$

  - where Q_i is $\exists$ when i is odd; is $\forall$ when i is even

$y_1, y_2, \ldots, y_i$ can be integers, strings, or any elements in countable sets



Arithmetical Hierarchy

- For i≥1, a language L is in $\mathbf{\Sigma_i}$ if there exists a TM M that always halts s.t.

  $x \in L \Leftrightarrow (\exists y_1)(\forall y_2)(\exists y_3)\ldots(Q_i y_i)[M(x,y_1,y_2,\ldots,y_i) \text{ accepts}]$

  - where Q_i is ∃ when i is odd; is ∀ when i is even

- For i≥1, a language L is in $\mathbf{\Pi_i}$ if there exists a TM M that always halts s.t.

  $x \in L \Leftrightarrow (\forall y_1)(\exists y_2)(\forall y_3)\ldots(Q_i y_i)[M(x,y_1,y_2,\ldots,y_i) \text{ accepts}]$

  - where Q_i is ∀ when i is odd; is ∃ when i is even

- Then $\mathbf{\Sigma_0 = \Pi_0}$ is the set of recursive languages



Arithmetical Hierarchy

- For $i \geq 1$, a language L is in $\mathbf{\Sigma_i}$ if there exists a TM M that always halts s.t.

  $x \in L \Leftrightarrow (\exists y_1)(\forall y_2)(\exists y_3)\ldots(Q_i y_i)[M(x,y_1,y_2,\ldots,y_i) \text{ accepts}]$

  - where Q_i is $\exists$ when i is odd; is $\forall$ when i is even

- For $i \geq 1$, a language L is in $\mathbf{\Pi_i}$ if there exists a TM M that always halts s.t.

  $x \in L \Leftrightarrow (\forall y_1)(\exists y_2)(\forall y_3)\ldots(Q_i y_i)[M(x,y_1,y_2,\ldots,y_i) \text{ accepts}]$

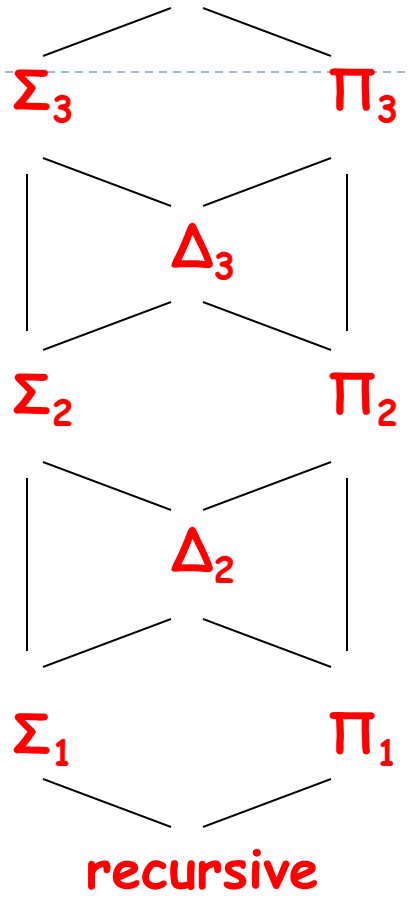
  - where Q_i is $\forall$ when i is odd; is $\exists$ when i is even

- Also, define $\mathbf{\Delta_i = \Sigma_i \cap \Pi_i}$



Arithmetical Hierarchy

- Theorem: Σ_1 is the set of recursive enumerable languages
  - L' is in Σ_1: $x \in L' \Leftrightarrow (\exists y)[(x,y) \in L]$ for some recursive language L
  - L' is r.e.: $x \in L' \Leftrightarrow$ M(x) accepts



Turing degrees are "partitioned" by Turing reductions, while arithmetical hierarchy are "partitioned" by mapping reductions



Arithmetical Hierarchy

- Theorem: Σ_1 is the set of recursive enumerable languages.
  - L' is in Σ_1: $x \in L' \Leftrightarrow (\exists y)[(x,y) \in L]$ for recursive language L
  - L' is r.e.: $x \in L' \Leftrightarrow$ M(x) accepts

- Proof.
  - given a TM M, define M'(x,s) for $s \in \mathbb{N}$ so that M'(x,s)=M(x) if M(x) halts in s steps, otherwise M'(x,s) rejects
  - so M'(x,s) always halts, and M(x) accepts $\Leftrightarrow (\exists s)$[M'(x,s) accepts]
  - Thus, if L' is r.e. , $x \in L' \Leftrightarrow$ M(x) accepts $\Leftrightarrow (\exists s)$[M'(x,s) accepts] $\Rightarrow$ L' is in Σ_1
  - If $x \in L' \Leftrightarrow (\exists y)[(x,y) \in L]$, define a machine N(x) for enumerating such y, then L' is r.e.

A_{TM} is in Σ_1
For all Σ_1 language L, $L \leq_m A_{TM}$
Thus, A_{TM} is Σ_1-complete



$$ALL_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

- Show: ALL_{TM} is in $\mathbf{\Pi}_2$



$$ALL_{\mathrm{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

- Show: ALL_{TM} is in $\mathbf{\Pi}_2$

- Proof.
  - which is $(\forall x)[M(x) \text{ accepts}] \Leftrightarrow (\forall x)(\exists s)[M'(x,s) \text{ accepts}]$



$$ALL_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

- Show: For all language L in $\mathbf{\Pi}_2$, L $\leq_m$ ALL_{TM}
  - Thus ALL_{TM} is $\mathbf{\Pi}_2$-complete



$$ALL_{TM} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

- Show: For all language L in Π_2, $L \leq_m ALL_{TM}$
  - Thus ALL_{TM} is Π_2-complete

  - $x \in L \Leftrightarrow (\forall y)(\exists z)[R(x,y,z) \text{ accepts}]$, where R always halts
  - given an x, construct a TM M_x:

```
Mx(y) {
        z=0;
        while (R(x,y,z) rejects)
                z++;
        accept;
}
```

if $x \in L$, $M_x(y)$ accepts all inputs y
if $x \notin L$, $(\exists y)(\forall z)[R(x,y,z) \text{ rejects}]$, so
M_x will loop on such y
so $x \in L \Leftrightarrow M_x \in ALL_{TM}$



$$ALL_{\mathrm{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

We have known $ALL_{TM} \in \mathbf{\Pi}_2$, what about:

$$E_{\mathrm{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts no inputs} \}$$



$$ALL_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

We have known $ALL_{TM} \in \mathbf{\Pi}_2$, what about:

$$E_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts no inputs} \}$$

In fact, $E_{TM} \in \mathbf{\Pi}_1$ (co-recognizable)
Since $M \in E_{TM} \Leftrightarrow (\forall x)(\forall s)[M(x)$ doesn't accept in s steps$]$



What about…

$$REC_{\mathrm{TM}} = \{\ \langle M \rangle\ : \mathrm{L}(M) \text{ is recursive}\}$$

- Note that M itself may loop, but the language it accepts can be accepted by a terminating M'
- Show $REC_{TM} \in \mathbf{\Sigma}_3$



$$ALL_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

$$E_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts no inputs} \}$$

$$REC_{\text{TM}} = \{ \langle M \rangle : \text{L}(M) \text{ is recursive} \}$$

- Show $REC_{TM} \in \Sigma_3$

- L(M) is recursive $\Leftrightarrow$ both L(M) and $\overline{\text{L(M)}}$ are recognizable
- so $\langle M \rangle \in REC_{TM} \Leftrightarrow (\exists N)[N \text{ is recursive and } L(M)=\overline{L(N)}]$

$\Leftrightarrow (\exists N)[N \text{ is recursive and } L(M) \cap L(N)=\emptyset \text{ and } L(M) \cup L(N)=\mathbb{N}]$

  - (we can construct machines $L(M \cap N) = L(M) \cap L(N)$ and $L(M \cup N) = L(M) \cup L(N)$)

$\Leftrightarrow (\exists N)[\text{“N halts”} \in ALL_{TM} \text{ and } (M \cap N) \in E_{TM} \text{ and } (M \cup N) \in ALL_{TM}]$



What about the SUPERHALT?

- SUPERHALT = {<M^{HALT}, w> | TM M^{HALT} halts on input w>}



What about the SUPERHALT?

- SUPERHALT = {<M^{HALT}, w> | TM M^{HALT} halts on input w>}

  - <M^{HALT}, w>∈SUPERHALT, iff there is an $s \in \mathbb{N}$, so that M^{HALT} halts on w in s steps
  - In these s steps, M^{HALT} can call the oracle $HALT_{TM}$ for ≤s times



  - remember $HALT_{TM}$ is in Σ_1, which is (∃y)[H(x,y) accepts]
  - so when the answer is yes, it means (∃y)[H(x,y) accepts]
  - when the answer is no, it means (∀z)[$\overline{H}$(x,z) accepts]



What about the SUPERHALT?

- SUPERHALT = $\{\langle M^{HALT}, w\rangle \mid \text{TM } M^{HALT} \text{ halts on input } w\}$

  - $\langle M^{HALT}, w\rangle \in$ SUPERHALT, iff there is an $s \in \mathbb{N}$, so that M^{HALT} halts on w in s steps
  - In these s steps, M^{HALT} can call the oracle $HALT_{TM}$ for $\leq s$ times



  - Thus $\langle M^{HALT}, w\rangle \in$ SUPERHALT, when

    $(\exists s)(\exists y_1)(\exists y_2)\ldots(\exists y_s)(\forall z_1)(\forall z_2)\ldots(\forall z_s)$ […]

  - when M^{HALT} calls the oracle $HALT_{TM}$, if the answer is yes, use a y_i, if the answer is no, use a z_i

We can encode a sequence of numbers to one natural number



What about the SUPERHALT?

- SUPERHALT = $\{\langle M^{HALT}, w\rangle \mid \text{TM } M^{HALT} \text{ halts on input } w\}$

THUS, SUPERHALT$\in\Sigma_2$

  - $\langle M^{HALT}, w\rangle \in$ SUPERHALT, iff there is an $s\in\mathbb{N}$, so that M^{HALT} halts on w in s steps
  - In these s steps, M^{HALT} can call the oracle $HALT_{TM}$ for $\leq s$ times



  - Thus $\langle M^{HALT}, w\rangle \in$ SUPERHALT, when

    $(\exists s)(\exists y_1)(\exists y_2)\ldots(\exists y_s)(\forall z_1)(\forall z_2)\ldots(\forall z_s)$ […]

  - when M^{HALT} calls the oracle $HALT_{TM}$, if the answer is yes, use a y_i, if the answer is no, use a z_i



The degree 0”

$$ALL_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

- It is easy to show that $ALL_{TM} \leq_T$ SUPERHALT



The degree 0”

$$ALL_{\mathrm{TM}} = \{\ \langle M \rangle\ : M \text{ is a TM that accepts all inputs}\}$$

- It is easy to show that $ALL_{TM} \leq_T$ SUPERHALT
  - suppose we have an A_{TM} oracle (equiv. to a $HALT_{TM}$ oracle)

```
N^HALT(y) {          //ignore y
        x=0;
        do {
                if (M, x)∈A_TM
                        x++;
                else
                        reject;
        while(true);
}
```

- So if (N^{HALT},w) for any w is not in SUPERHALT, M is in ALL_{TM}



The degree 0”

$$ALL_{\text{TM}} = \{ \langle M \rangle : M \text{ is a TM that accepts all inputs} \}$$

- We can also show SUPERHALT $\leq_T$ ALL_{TM}



The degree 0”

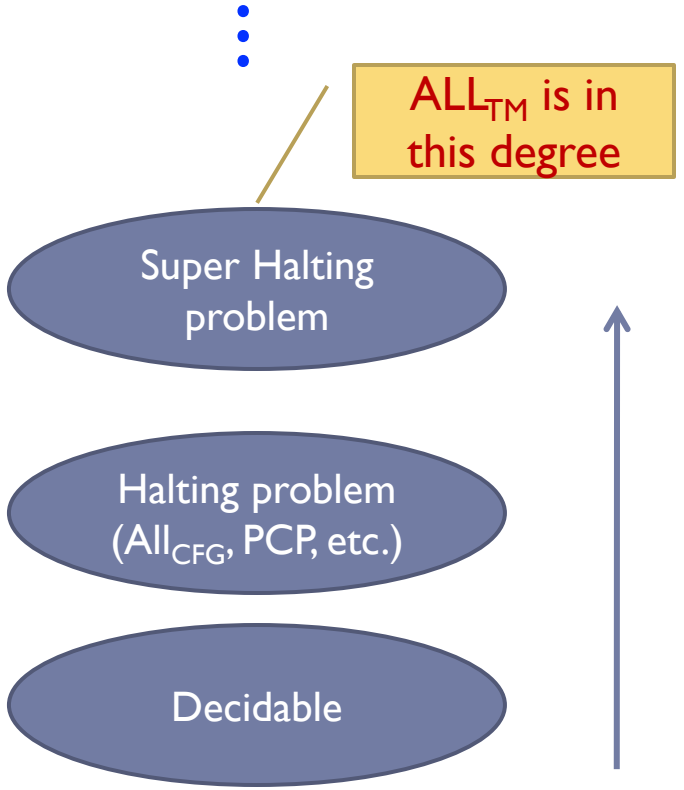
$$ALL_{\mathrm{TM}} = \{\ \langle M \rangle\ : M \text{ is a TM that accepts all inputs}\}$$

- How to show SUPERHALT $\leq_T$ ALL_{TM} ?
- Proof.
  - We have shown that for all language L in $\mathbf{\Pi}_2$, L $\leq_m$ ALL_{TM}
  - so if we have an oracle for ALL_{TM}, we can decide all languages in $\mathbf{\Pi}_2$
  - But SUPERHALT is in $\mathbf{\Sigma}_2$, so we can also decide SUPERHALT



Turing degree

- A Turing degree is such an equivalent class
  - 0: decidable problems
  - 0': problems equivalent to the Halting problem
  - 0'':problems equivalent to the SUPERHALT problem
  - 0''': SUPERSUPERHALT
  - …

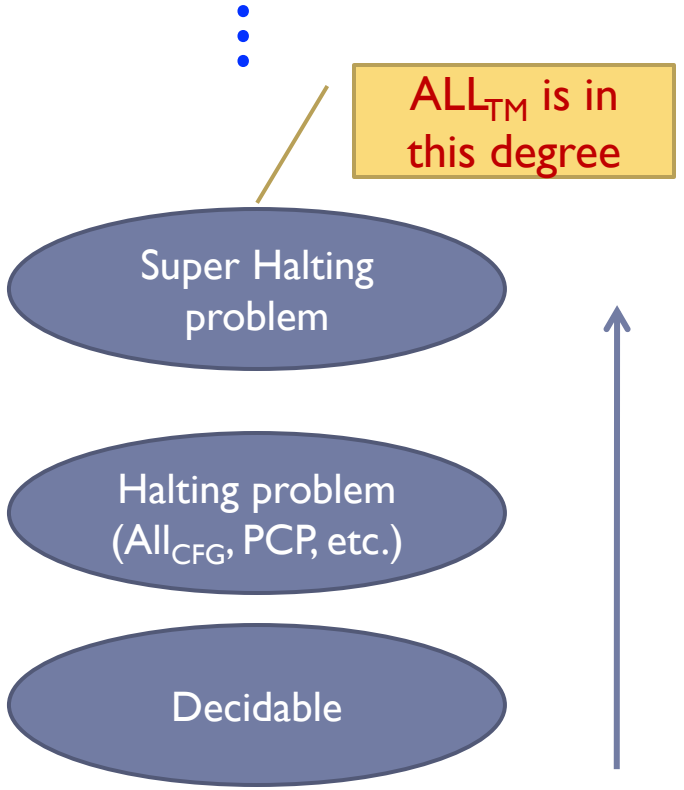




Turing degree

- In fact, A' denotes the degree of the problem $HALT^A$:
  - $\{<M^A, w> \mid \text{TM } M^A \text{ halts on input } w>\}$
  - "Turing jump"
  - HALT: $\emptyset'$
  - SUPERHALT: $\emptyset''$
  - SUPER…SUPERHALT: $\emptyset^{(n)}$

- Post's theorem: $\emptyset^{(n)}$ is $\mathbf{\Sigma}_n$-complete



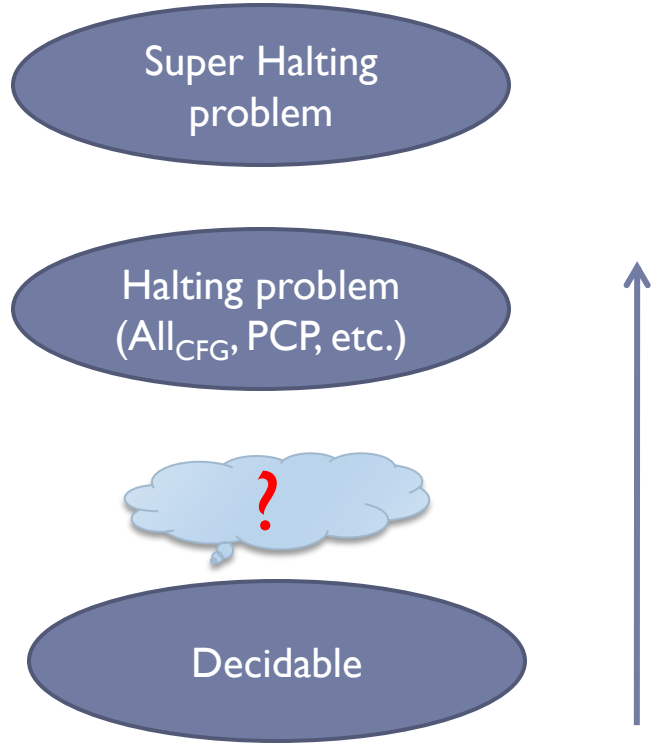


Turing degree

Is there any Turing degree between HALT and decidable?

any problem:

1. not decidable
2. reducible to HALT
3. not equivalent to HALT





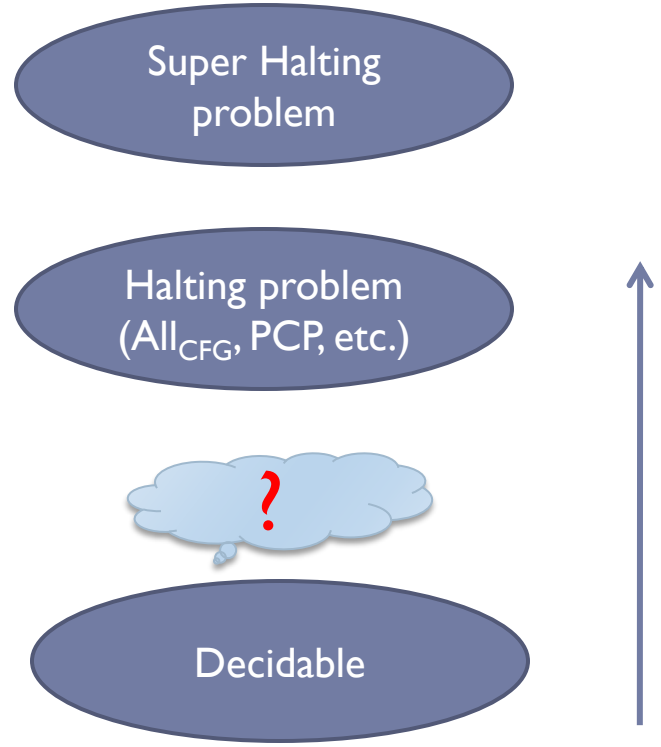
Turing degree

Is there any Turing degree between HALT and decidable?

Yes

This was called "Post's problem", first found in 1944

In fact, there are infinite many such degrees





Summary

- Reduction via computation history
- Mapping reducibility
- Recursion theorem
- Kolmogorov complexity
- Turing degree and arithmetical hierarchy

- Office hour: 4:00-5:00pm, every Monday
- Tutorial session?

